CPU Scheduling Algorithms

Laboratory Report

Date: Thursday 04/23/2026
Course: Operating Systems
Instructor: Dr. Hanene Rouainia

1. Introduction

CPU scheduling is a core function of any operating system. The scheduler determines which process in the ready queue gets the CPU computational resource (execution) at any given time. The choice of scheduling algorithm directly impacts system performance in terms of **turnaround time**, **waiting time**, and **response time**.

This report documents the design, implementation, and comparative analysis of seven CPU scheduling algorithms written in C:

- **FCFS** — First-Come First-Served (Non-preemptive)
- **SJF** — Shortest Job First (Non-preemptive)
- **SRTF** — Shortest Remaining Time First (Preemptive)
- **Round Robin** — Time-sliced preemptive scheduling (quantum = 2)
- **Priority Scheduling** — Non-preemptive variation
- **Priority Scheduling** — Preemptive variation
- **Multilevel Queue** — Fixed partitioned queues with per-queue algorithms

All algorithms operate on the same processes list to allow direct comparison across metrics (waiting time, response time..etc).

2. Test Process Set

All algorithms were tested against the following four processes:

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	3	3
P3	2	8	1
P4	4	2	2

3. Implementation Methodology

All algorithms are implemented in C and share common infrastructure: a **Queue** (linked list, FIFO) defined in **queue.h/queue.c**, a **MinHeap** (binary min-heap) defined in **priority_queue.h/priority_queue.c**, and a shared **print_results()** utility in **utils.c**. Original process data is never mutated — each algorithm works on a heap-allocated copy so results are isolated.

3.1 First-Come First-Served (FCFS)

FCFS is the simplest non-preemptive algorithm. Processes are inserted into a FIFO **ready queue** in arrival order (arrival time). The CPU dequeues one process, runs it to completion, then moves to the next.

```
Gantt Chart:
| P1 | P2 | P3 | P4 |
0    5    8    16   18

═══════════════════ FCFS ═══════════════════

P   | AT | BT | CT | TAT | WT | RT
──────────────────────────────────────
P1  | 0  | 5  | 5  | 5   | 0  | 0
──────────────────────────────────────
P2  | 1  | 3  | 8  | 7   | 4  | 4
──────────────────────────────────────
P3  | 2  | 8  | 16 | 14  | 6  | 6
──────────────────────────────────────
P4  | 4  | 2  | 18 | 14  | 12 | 12
──────────────────────────────────────
Average TAT = 10.00
Average WT  = 5.50
Average RT  = 5.50
```

t: FCFS results table]

3.2 Shortest Job First (SJF — Non-preemptive)

SJF selects the process with the shortest **Execution time** among all currently available

processes. It is non-preemptive once a process starts, it runs to completion. If two processes have equal execution time the one with the minimum arrive time gets executed.

The implementation uses a **MinHeap** ordered by execution time . Processes are pushed onto the heap as they arrive. On each iteration the shortest available job is popped and run to completion.

```
Gantt Chart:
| P1 | P4 | P2 | P3 |
0    5    7    10   18

================ SJF (Non-Preemptive) ===

P   | AT | BT | CT | TAT | WT | RT
-----------------------------------
P1  | 0  | 5  | 5  | 5   | 0  | 0
-----------------------------------
P2  | 1  | 3  | 10 | 9   | 6  | 6
-----------------------------------
P3  | 2  | 8  | 18 | 16  | 8  | 8
-----------------------------------
P4  | 4  | 2  | 7  | 3   | 1  | 1
-----------------------------------
Average TAT = 8.25
Average WT  = 3.75
Average RT  = 3.75
```

3.3 Shortest Remaining Time First (SRTF — Preemptive)

SRTF is the preemptive version of SJF. The CPU runs the process with the shortest **remaining execution time**. A newly arrived process can preempt the currently running process if its execution time is shorter.

The implementation reuses the same **MinHeap** as SJF.

```
Gantt Chart:
| P1 | P4 | P2 | P3 |
0    5    7    10   18

================ SJF (Non-Preemptive) ===

P   | AT | BT | CT | TAT | WT | RT
-----------------------------------
P1  | 0  | 5  | 5  | 5   | 0  | 0
-----------------------------------
P2  | 1  | 3  | 10 | 9   | 6  | 6
-----------------------------------
P3  | 2  | 8  | 18 | 16  | 8  | 8
-----------------------------------
P4  | 4  | 2  | 7  | 3   | 1  | 1
-----------------------------------
Average TAT = 8.25
Average WT  = 3.75
Average RT  = 3.75
```

3.4 Round Robin (Preemptive, with Quantum)

Round Robin is primarely used in time shared systems where it assigns each process a fixed **time quantum**. If a process does not complete within its quantum, it is preempted and placed at the rear of the ready queue.

The implementation uses the FIFO **Queue** . An **input_idx** pointer walks the processes array and enqueues processes as they arrive. Each iteration dequeues one process, runs it for **min(quantum, remaining)** units, then enqueues any new arrivals before re-queuing the current process if it has remaining work. CPU idle time is resolved by jumping **current_time** directly to **results[input_idx].arrival_time (so that the CPU remains busy and the ready queue doesn't stay empty)**.

```
Gantt Chart:
| P1 | P2 | P3 | P1 | P4 | P2 | P3 | P1 | P3 | P3 |
0    2    4    6    8    10   11   13   14   16   18

================= Round Robin =================

P   | AT | BT | CT | TAT | WT | RT
-----------------------------------
P1  | 0  | 5  | 14 | 14  | 9  | 0
-----------------------------------
P2  | 1  | 3  | 11 | 10  | 7  | 1
-----------------------------------
P3  | 2  | 8  | 18 | 16  | 8  | 2
-----------------------------------
P4  | 4  | 2  | 10 | 6   | 4  | 4
-----------------------------------
Average TAT = 11.50
Average WT  = 7.00
Average RT  = 1.75

================================================
```

3.5 Priority Scheduling — Non-preemptive

Non-preemptive Priority Scheduling selects the highest-priority process from the ready queue.

Once a process occupies the CPU it runs to completion. Lower numeric value = higher priority (convention used in this implementation).

The implementation uses the **MinHeap** ordered by priority field instead of burst time. The processes are pushed onto the heap, and the minimum-priority process is popped and executed to completion each cycle.

```
═══════════════ Priority (Non-Preemptive) ═══════════════

P   | AT | BT | CT | TAT | WT | RT
──────────────────────────────────
P1  | 0  | 5  | 5  | 5   | 0  | 0
──────────────────────────────────
P2  | 1  | 3  | 18 | 17  | 14 | 14
──────────────────────────────────
P3  | 2  | 8  | 13 | 11  | 3  | 3
──────────────────────────────────
P4  | 4  | 2  | 15 | 11  | 9  | 9
──────────────────────────────────
Average TAT = 11.00
Average WT  = 6.50
Average RT  = 6.50

═══════════════════════════════════════

Gantt Chart:
| P1 | P2 | P3 | P1 | P4 | P2 | P3 | P1 | P3 | P3 |
0    2    4    6    8    10   11   13   14   16   18
```

3.6 Priority Scheduling — Preemptive

The preemptive variant re-evaluates the ready queue each time, similar to SRTF. A newly arrived higher-priority process immediately preempts the running process.

The implementation runs 1 unit at a time, pops the highest-priority process from the heap, decrements its remaining time, then pushes it back if unfinished.

```
═══════════════ Priority (Preemptive) ═══════════════

P   | AT | BT | CT | TAT | WT | RT
──────────────────────────────────
P1  | 0  | 5  | 13 | 13  | 8  | 0
──────────────────────────────────
P2  | 1  | 3  | 18 | 17  | 14 | 14
──────────────────────────────────
P3  | 2  | 8  | 10 | 8   | 0  | 0
──────────────────────────────────
P4  | 4  | 2  | 15 | 11  | 9  | 9
──────────────────────────────────
Average TAT = 12.25
Average WT  = 7.75
Average RT  = 5.75

═══════════════════════════════════

Gantt Chart:
| P1 | P3 | P4 | P2 |
0    5    13   15   18
```

3.7 Multilevel Queue

Multilevel Queue Scheduling partitions the ready queue into several **fixed queues**, each with its own scheduling algorithm. Processes are permanently assigned to a queue based on their type. The scheduler services higher-priority queues first , lower queues only get CPU time when all higher queues are empty.

The implementation maintains an array of queues. At each scheduling decision the algorithm scans from the highest-priority queue downward and applies that queue's local algorithm (FCFS, SJF in this case.) to select the next process.

```
Gantt Chart:
| P1 | P2 | P4 | P3 |
0    5    8    10   18

═══ Multilevel Queue Scheduling ═══
Number of queues: 2
Queue 0: priority = 0, algorithm = FCFS
Queue 1: priority = 1, algorithm = SJF

P   | AT | BT | CT | TAT | WT | RT | Queue
P1  | 0  | 5  | 5  | 5   | 0  | 0  |  Q0
P2  | 1  | 3  | 8  | 7   | 4  | 4  |  Q0
P3  | 2  | 8  | 18 | 16  | 8  | 8  |  Q1
P4  | 4  | 2  | 10 | 6   | 4  | 4  |  Q1

Average Turnaround Time = 8.50
Average Waiting Time    = 4.00
Average Response Time   = 4.00
```

[Terminal output screenshot: Multilevel Queue results table]

5. Comparative Analysis

5.1 Throughput and Turnaround Time

Throughput Definition*:* The number of processes executed by the CPU in a time unit.

SRTF achieves the lowest average turnaround time because it is provably optimal for minimizing average waiting time among all non-preemptive and preemptive variants. it always schedules the process closest to completion. F**CFS** suffers from the **convoy effect,** a long process at the front of the queue delays all shorter ones behind it.

5.2 Waiting Time

SJF minimizes average waiting time among non-preemptive algorithms because it always selects the shortest job, leaving longer jobs to wait while short jobs finish quickly. SRTF extends this to the preemptive case. Round Robin distributes waiting time more evenly across processes, which raises the average but reduces the variance.

5.3 Response Time and Fairness

Round Robin achieves the lowest average **response time** because every process receives CPU time within at most **(n-1) × quantum** units of arriving. This makes it well suited for **interactive systems** where a fast initial response matters more than minimizing total completion time. FCFS and SJF can produce very high response times for processes that arrive late or have long predecessors.

5.4 Priority Scheduling

Priority scheduling introduces **process importance** as a scheduling criterion. Non-preemptive priority is straightforward but can cause **starvation** of low-priority processes. Preemptive priority guarantees that critical processes receive the CPU immediately on arrival but increases **context switch overhead**.

5.5 Multilevel Queue

Multilevel Queue Scheduling is designed for systems where processes fall into distinct classes (e.g., system processes, interactive processes, batch jobs). Each class gets its own queue and algorithm, providing fine-grained control. The main trade-off is **rigidity,** a process cannot move between queues, so misclassification leads to poor performance.

6. Conclusion

The choice of CPU scheduling algorithm should reflect the workload type and system goals:

- **FCFS**: Simple and predictable, appropriate for **batch systems** where all jobs have similar burst times and fairness concerns are secondary.

- **SJF / SRTF**: Optimal for minimizing average waiting time in **batch environments** where burst times are known in advance. SRTF provides better performance at the cost of more frequent context switches.
- **Round Robin**: Best suited for **time-sharing and interactive systems** where responsiveness and fairness are the primary objectives. The quantum size is a critical tuning parameter too small increases context switch overhead; too large degrades response time.
- **Priority Scheduling**: Appropriate for **real-time or mixed-criticality systems** where certain processes must be serviced before others. Starvation must be addressed explicitly through aging or other mechanisms.
- **Multilevel Queue**: Suitable for **general-purpose operating systems** that serve heterogeneous workloads requiring different scheduling policies per process class.

No single algorithm is universally optimal. Real-world schedulers (e.g., Linux CFS) combine ideas from multiple policies to balance throughput, fairness, and responsiveness across diverse workloads.

7. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*, 10th ed. Wiley.